

Music Genre Classification

Neural Networks Final Project



Project Goals.....	2
I. Dataset Selection and Description.....	2
II. Data Extraction and Pre-processing Workflow.....	3
Selection of the audio recordings used.....	3
Audio Data Extraction.....	4
TFRecord Generation.....	4
III. Model generation and training.....	5
Data Processing and Modeling Framework.....	5
Optimization Strategies.....	6
Traceability, reproducibility, and model storage.....	7
IV. Inference pipeline.....	8
Overall functioning of the pipeline.....	8
Automatic generation of interpretable reports.....	9
V. Model performance evaluation.....	9
Global Accuracy and Prediction Imbalance.....	9
Disparities Between Classes.....	10
Dominant Confusion Patterns.....	10
Perspectives for Improvement.....	11
Conclusion, outlook and additional insights.....	12

Project Goals

The goal of this project is to design a neural network model capable of automatically classifying different musical excerpts into several genres (for example: Pop, Rock, Electronic, Classical, etc.). As a pianist, this topic is especially motivating for me because it connects my interests in both music and computer science. We will work through a complete processing pipeline: first, we will select an appropriate dataset, then we will carry out preprocessing on the raw audio signals. Finally, we will build a classification model and analyze its performance in order to assess the quality of our approach.

In this project, the main objective is not to achieve the highest possible performance, but rather to learn how to design end-to-end pipelines for data processing and model training. The results remain partly limited and variable, since we are working with public datasets whose quality is not guaranteed and with relatively modest hardware resources.

I. Dataset Selection and Description

The dataset selected for this project is FMA Medium (Free Music Archive). It was chosen for its methodological rigor and the richness of its metadata: each recording comes with detailed information about the genre, sub-genres, artist, title, and official splits. Unlike some older datasets such as GTZAN, it avoids several well-known issues (labeling errors, duplicate tracks, and leakage between training and test sets).

The music data that you can find on Kaggle was created by Michaël Defferrard, Kirell Benzi, Pierre Vanderghyest and Xavier Bresson. They showed this music data at the ISMIR 2017 conference. This music data has more than 10,000 audio clips in MP3 format. Each audio clip is 30 seconds long. There is also a CSV file that has information about every track. This CSV file is like a map that helps us keep track of the music data as we do our experiments, with the music data.

FMA also provides pre-computed audio features. However, in the first stage, we will focus on extracting our own features directly from the raw signals, in order to better understand and control the entire processing pipeline. Depending on how the project evolves, we may later compare our results with those obtained using the provided features if needed.

Link to the dataset on Kaggle:

<https://www.kaggle.com/datasets/imsparsh/fma-free-music-archive-small-medium>

In this dataset, two main components are used:

- **fma_medium**: contains all the audio files organized into different subfolders.
- **metadata**: mainly contains the file *tracks.csv*, which links the audio file paths to their IDs and labels, along with additional information such as the title, artist, and more.

These two folders were downloaded into the project's *data* directory and will be the only external resources used throughout the project.

II. Data Extraction and Pre-processing Workflow

The preprocessing pipeline is organized into two main stages. The first stage focuses on preparing the metadata (metadata folder), including the selection of the main genres, class balancing, and validation of the audio files. The second stage involves transforming the audio data, from loading and segmenting the tracks to extracting spectral representations and generating TFRecord files.

Selection of the audio recordings used

Because the dataset is particularly large, a filtering step was performed in order to remove information considered non-essential for the classification task. To simplify the problem, only the six most represented musical genres were kept: Experimental, Hip-Hop, Folk, Electronic, Pop, and Rock. Class balancing was carried out through undersampling to ensure that each genre contained the same number of tracks, thereby avoiding biases caused by data imbalance.

In total, around 14,000 clips of 30 seconds each (about 2,300 per class) were selected, which represents a data volume assumed to be sufficient for training. Each validated track was assigned a numeric label according to a fixed mapping, and absolute audio paths were generated to simplify further loading. Missing or corrupted files were removed to preserve dataset integrity.

All selected tracks were stored in a CSV file named *filtered_tracks.csv* inside the metadata folder, containing for each entry the pair *track_id* – *genre*. Additional intermediate CSV files were also saved, including *label_mapping.csv*, which lists the genres and their corresponding mapping and will be used during inference.

Audio Data Extraction

The MP3 files from the *FMA_medium* folder were converted to mono signals with a sampling rate of 22,050 Hz. This choice is somewhat arbitrary, but it is justified by the spectral coverage required for musical analysis while also reducing computational cost. Each 30-second clip was then divided into 4-second windows with 50% overlap. This segmentation acts as a natural form of data augmentation, helping improve model training.

The raw signals were transformed into representations suitable for convolutional neural networks by computing log-Mel spectrograms. Each segment underwent a Short-Time Fourier Transform (STFT) using a 2048-sample window and a hop length of 512 samples. The resulting frequencies were projected onto 128 Mel bands, then converted to a logarithmic decibel scale, limited to the range [-80 dB, 0 dB]. This two-dimensional representation ($128 \times \sim 130$) captures the main harmonic and rhythmic characteristics needed for genre classification while remaining compact for learning. Additional experiments with 64 and 32 Mel bands were also carried out in order to reduce computation time during training. All extraction parameters (sampling rate, segment length, etc.) are stored in a file called *cste.py*, and can therefore be modified.

Normalization was applied at the dataset level, using statistics computed only from the training set. Each frequency band was normalized independently to preserve natural energy differences between low and high frequencies. Z-score standardization was used so that values are centered around zero with unit variance. The normalization parameters were saved in a file named *normalization.json*, ensuring consistency and reproducibility during inference.

TFRecord Generation

The dataset was split into training, validation, and test sets following a 70/15/15 ratio. Stratification was performed at the track level, ensuring that each split preserved the same genre distribution. This also prevents data leakage by guaranteeing that segments from the same track never appear in different subsets.

To optimize training of deep networks, the normalized segments were serialized into the TFRecord format. Each TFRecord file contains all segments from a single track, along with its label and the spectrogram dimensions. The directory structure clearly separates the train, validation, and test sets, and processing was parallelized across all available CPU cores. Although this approach is less efficient than using a GPU, the computer used for this project does not have TensorFlow-compatible GPU hardware, and attempts to rely on Google Colab turned out to be cumbersome due to data loading and audio file management constraints.

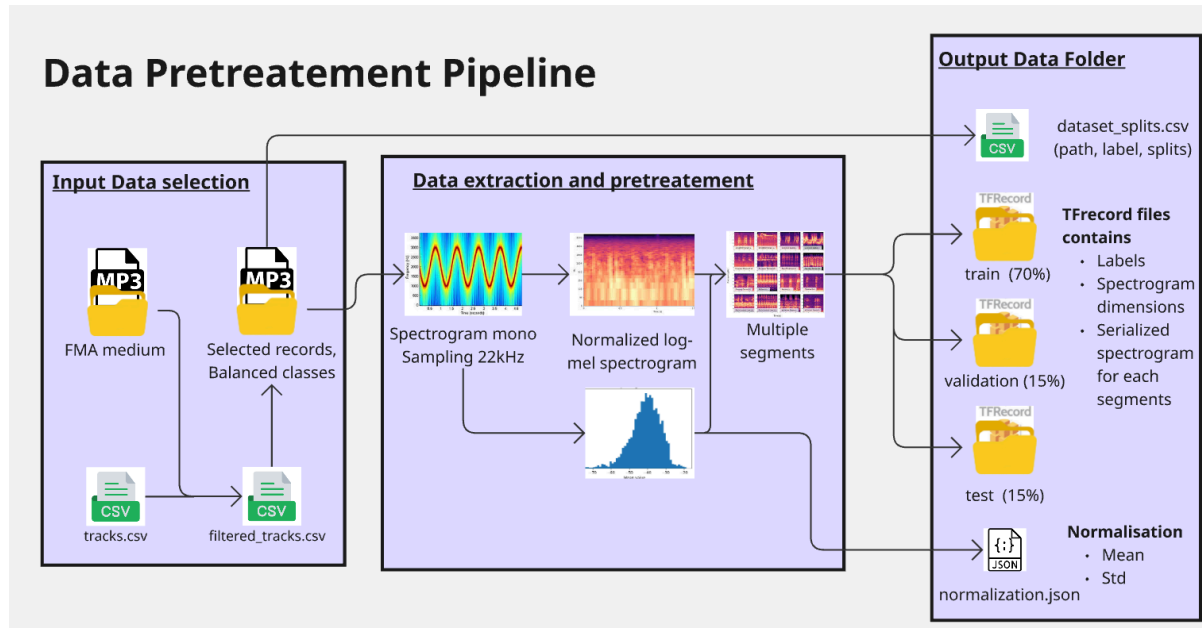


Figure 1: Summary of the data pipeline, from the selection of relevant audio recordings to the generation of the final dataset, including preprocessing and feature extraction.

III. Model generation and training

Data Processing and Modeling Framework

Data management relies on the TFRecord format, chosen for its efficient sequential access and tight integration with the TensorFlow environment. Training is performed sequentially for each TFRecord, which prevents data leakage. The use of a GlobalMaxPooling layer establishes translation invariance and reduces the number of parameters, which lowers the risk of overfitting, improves model flexibility, and decreases computation time.

The pipeline primarily uses convolutional neural network (CNN) architectures. CNNs are particularly suited to time–frequency representations (log-mel spectrograms), as they explicitly exploit local correlations across both frequency and time domains, allowing the detection of characteristic patterns such as harmonic textures or rhythmic regularities. In this project, the models presented in the code should not be interpreted as fixed or unique architectures; they serve as illustrative points within a broader design space, where many models are generated, tested, evaluated, and automatically selected during the optimization process. The overall approach is therefore less about fixing a precise architecture and more about defining a scientifically coherent family of models aligned with the problem structure, then systematically exploring this family under memory, training stability, and generalization

constraints. This strategy combines methodological robustness, experimental reproducibility, and progressive adaptation of the model to the actual characteristics of the dataset.

Optimization Strategies

Optimization follows a hybrid strategy combining random search and neuroevolution. Random search ensures broad, unbiased coverage of the search space, which is defined by intervals of parameters, including both model architecture parameters (number of layers, number of neurons per layer, activation functions) and hyperparameters (learning rate, dropout, etc.). Training is partial, with a limited number of epochs used to estimate the potential of each architecture early while preserving computational resources.

The evolutionary phase uses hierarchical mutations. At each generation, multiple variations of a selected model are created by introducing mutations—of varying impact—within the search space. The best models are then selected, with evaluation based on validation accuracy. These operations are repeated, gradually steering the overall model population toward an optimum.

An early stopping mechanism halts training when the model stops learning after a set number of epochs, reducing unnecessary computation. It is important to note that all models from each generation, as well as those from the random search, are evaluated after partial training, with a relatively small number of epochs (chosen by the user at the start of the pipeline). This approach minimizes computation time, so only the final model is trained over a large number of epochs.

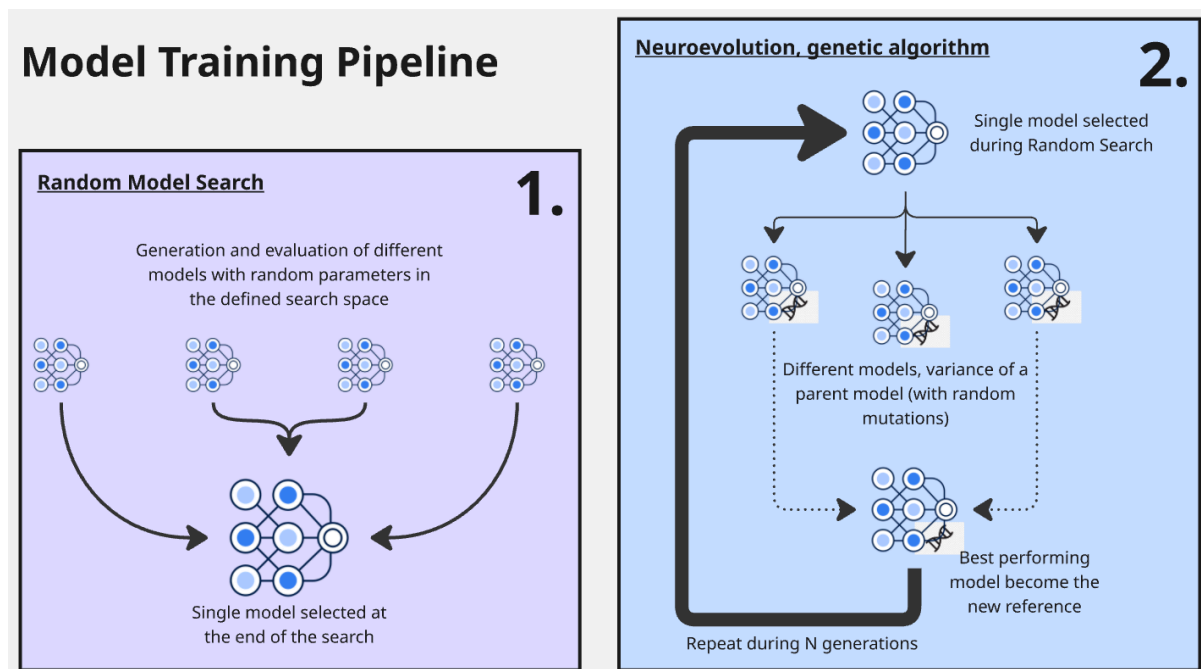


Figure 2: Summary of the optimization strategies used for convolutional models. A Random Search is first employed to broadly explore the search space, followed by Neuroevolution for fine-tuning the best-performing model.

Traceability, reproducibility, and model storage

The pipeline produces comprehensive documentation. Every configuration, metric, and saved model is recorded, and their performance is logged in a dedicated CSV file called *models_perfs.csv* to ensure traceability of the optimization process. An execution report is also generated, serving as a log and summarizing all parameters used during execution: the number of models generated at each step, model performance per generation, structures and paths of training data, etc.

This modular design opens possibilities for extension. The integration of more complex mechanisms is possible without major modifications to the infrastructure. There are potential avenues to further reduce computational costs, such as weight inheritance, progressive training, or predictive performance models, but these were not implemented in this project, which focused on the development and training of neural networks.

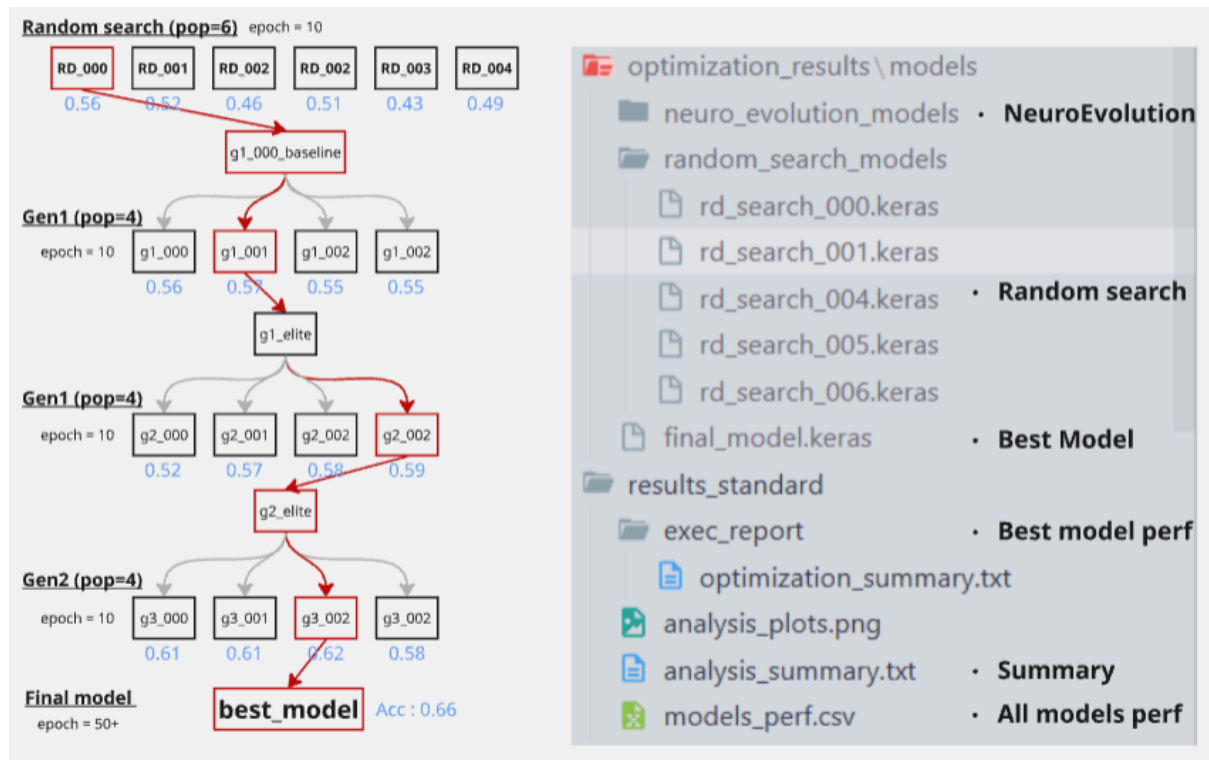


Figure 3: On the left, an example of execution for 3 generations and a random search, where the best-performing model is selected at each step. On the right, an example of the output folder after execution, containing the models in the *models* folder and the various performance reports in the *results_standards* folder.

IV. Inference pipeline

Overall functioning of the pipeline

The inference pipeline was designed to reproduce the exact conditions encountered during training while offering great flexibility in the format of input audio files. The inference pipeline always starts with a raw audio file. This audio file undergoes the same preprocessing and feature extraction steps applied to the training data. This processing is made possible by using a normalization file generated during the creation of the training dataset for a model. This file contains all the preprocessing operations applied to the training data, which allows the pipeline to provide the model with audio processed in exactly the same way during inference. By loading these values directly from the generated normalization file rather than defining them manually, the pipeline avoids any divergence between training and inference.

Once the audio is split into overlapping segments, a prediction is made for each segment. Then, the results are combined using a method that calculates the average of the probabilities, commonly called soft voting. This method is useful because it considers all segments, not just the one with the highest confidence. It provides a more stable estimate that better represents the input audio. It is important to note that in music, songs often have different sections such as verses, choruses, and instrumental bridges. These sections can have very different sounds and styles. Additionally, each piece may lean more toward one style than another, but this notion remains highly subjective. Soft voting allows focusing on a prediction across a set of styles with probabilities per style (for example, 54% Rock and 30% Pop) without forcing the music into a strictly defined genre. The pipeline was later extended to allow automatic processing of multiple files at once; the user then provides a folder name instead of a specific file. For each audio file processed, a detailed report is generated.

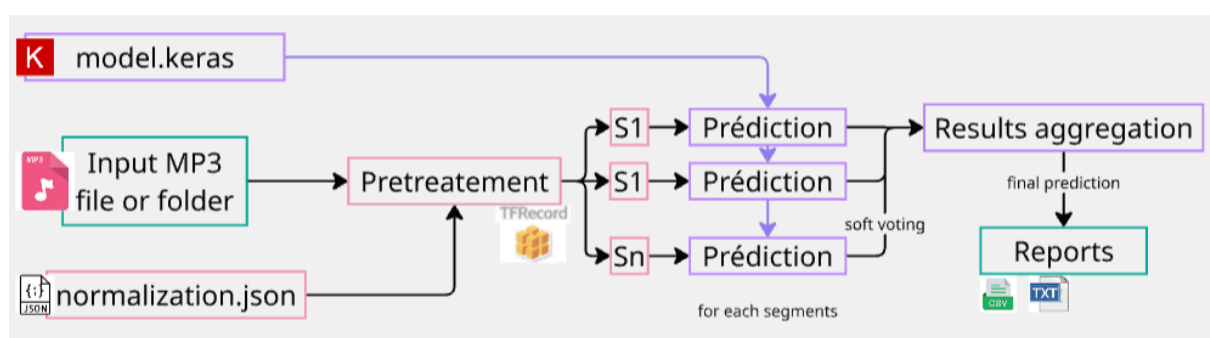


Figure 4: Overview of the inference pipeline - To run inference, at least one MP3 file, a normalization file, and a trained model must be provided. The MP3 file(s) are preprocessed and normalized, producing a TFRecord file that contains multiple segments. A prediction is then made for each segment, and the final prediction is computed using soft voting. Prediction reports are generated for each file.

Automatic generation of interpretable reports

Each prediction generates a comprehensive report that details the configuration used for the audio file, the number of segments analyzed, the predicted class, the associated confidence, and the complete probability distribution. To facilitate interpretation, probabilities are represented with text-based bars, providing a visual comparison of the likelihoods across classes. For multiple audio files, the pipeline automatically produces a summary CSV named *batch_prediction_summary*, which consolidates the results by listing each file's path, the majority predicted class with its confidence, and a reference to the corresponding detailed report. By combining segment-level analysis, probabilistic aggregation across segments, and structured reporting, the pipeline maintains both accuracy and reproducibility, making it well-suited for large-scale evaluation of diverse musical datasets.

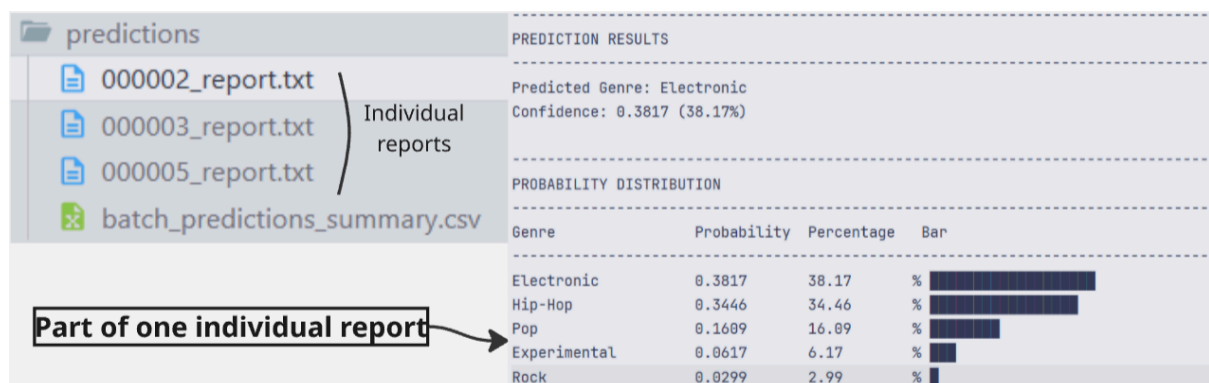


Figure 5: On the left, an example of a prediction folder generated for three audio recordings processed during inference. The individual reports for each evaluated audio file are visible, along with a CSV file containing overall metrics for each file. On the right, an example of a report file (.txt) is shown, displaying only the section related to the prediction result.

V. Model performance evaluation

Global Accuracy and Prediction Imbalance

The model reaches an overall accuracy of 56.89% on a test set composed of 450 audio files evenly distributed across six musical genres. This moderate performance can be explained by a critical imbalance in the system's predictive behavior. While the test data are perfectly balanced, with 75 samples per class, the distribution of predictions reveals a pronounced systematic bias: the Folk class is heavily over-predicted with 117 occurrences, whereas the Pop class is dramatically under-represented with only 29 predictions. This imbalance represents one of the model's major weaknesses and largely accounts for the significant performance variations observed across classes.

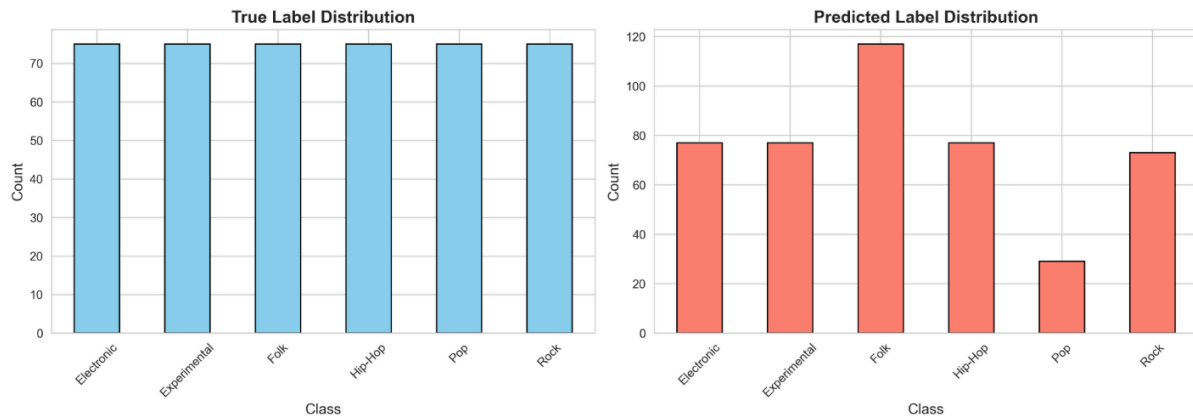


Figure 6: Class distribution in the test set on the left (in blue) shows that the dataset is balanced. On the right, the distribution of the predicted classes on this same set illustrates the model's tendency to over-predict the Folk class and under-predict the Pop class.

Disparities Between Classes

Class-wise analysis reveals performance gaps that deserve particular attention. The Rock and Hip-Hop genres stand out, with high F1 scores of 0.7297 and 0.7105, indicating that the model has successfully captured strong acoustic signatures for these categories. In contrast, the Pop class shows a critical failure, with an F1 score of only 0.3077 and a recall of 0.2133, meaning that nearly 80 percent of true Pop tracks are not correctly identified. This difficulty in predicting Pop may partly stem from the strong stylistic diversity found in contemporary Pop music, whereas the other studied genres are more clearly defined and therefore easier to recognize. Finally, the Folk class exhibits the highest recall of all categories (0.7867) but relatively poor precision (0.5043). This pattern suggests that the model detects genuine Folk tracks effectively, but at the cost of many false positives, most of which originate from the Pop class.

Dominant Confusion Patterns

Examining the confusion matrix reveals systematic error trends, which provide valuable insight into the model's limitations. The most frequent confusion occurs when Pop tracks are incorrectly classified as Folk. This happens 33 times, representing 44% of all Pop samples in the test set. This substantial error once again explains why the recall for Pop is low and why there are too many Folk predictions. A second pattern appears more subtly in confusions among genres that rely on electronic textures. The Electronic and Experimental classes show strong bidirectional confusion, as do Hip-Hop and Electronic. These confusions suggest genuine overlap within the acoustic feature space, particularly in cases where contemporary Hip-Hop relies on electronic production or Experimental music draws heavily on synthetic sound textures.

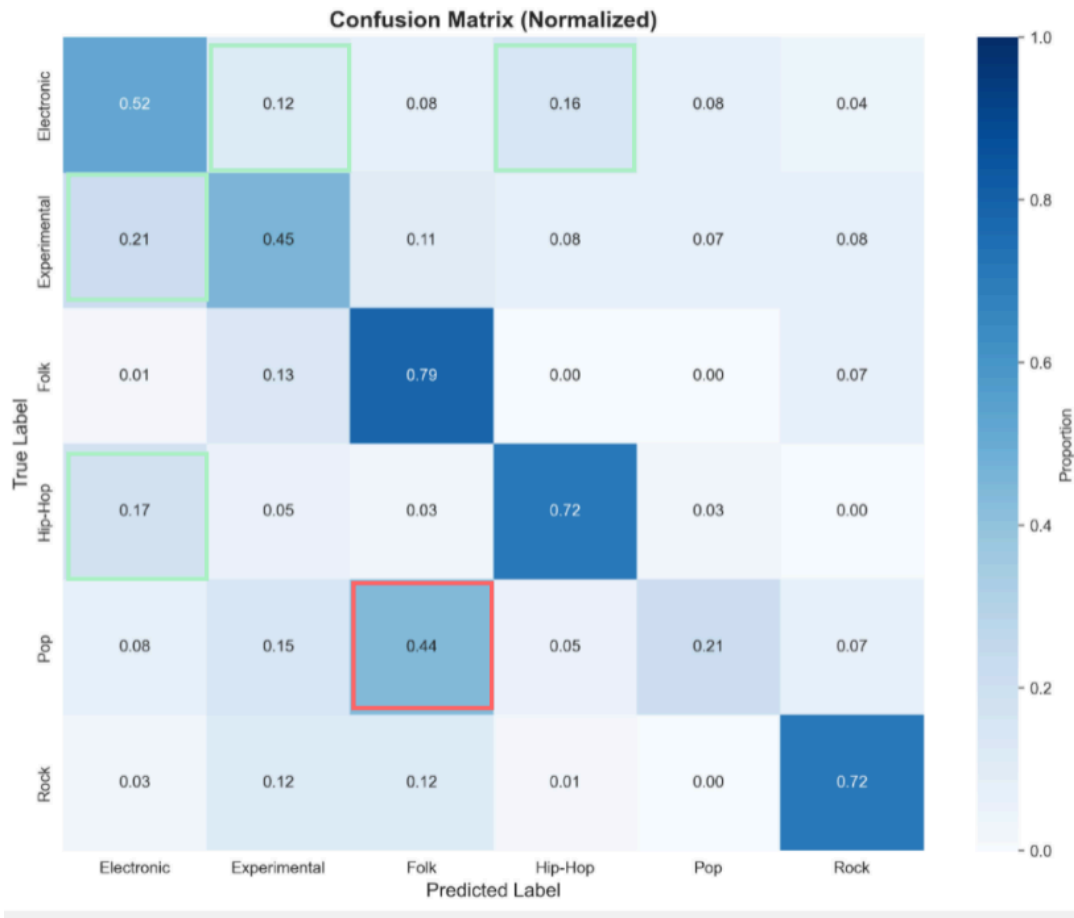


Figure 7: Highlighted in red, the confusion matrix reveals a strong confusion between the Pop and Folk classes, which is responsible for a drastic drop in model performance. In green, a more marginal confusion can be observed among genres that share electronic sound characteristics.

Perspectives for Improvement

This comprehensive evaluation reveals a music-genre classification model with uneven performance across categories, with an overall accuracy of 56.89%, which remains insufficient for any demanding real-world deployment.

Several avenues for improvement emerge from this analysis. First, rebalancing the training data and using class-weighting techniques could help mitigate the prediction bias toward Folk. Second, enriching the dataset with a greater variety of Pop examples could enable the model to better capture the variability within this genre. Third, implementing architectures capable of modeling long-term temporal dependencies between segments — such as recurrent networks or attention mechanisms — may improve prediction consistency, since the segments belonging to the same track are currently processed independently. Finally, exploring hierarchical learning strategies, in which macro-categories are detected first before refining the classification, could help better manage the systematic confusions observed between similar genres.

Conclusion, outlook and additional insights

This project gave me the opportunity to explore every aspect of a music-classification system, from the initial data collection stage to the final model evaluation. Working with the Free Music Archive was particularly instructive. I learned how to handle real audio files, how to build appropriate spectral representations, and how to design an optimization system combining random search and neural evolution. Developing an automated prediction pipeline with detailed reporting was also an interesting challenge. This project strengthened my understanding of CNN architectures applied to audio signal processing and exposed me to the practical constraints of machine learning: limited computational resources, variable data quality, and the need for traceability.

There are several ways this system could be improved. First, we could take advantage of the hierarchical structure of the FMA dataset, which includes 16 parent genres and 163 subgenres. This would allow us to implement multi-level classification: we could first identify the main genre, then refine the prediction by considering specific subgenres such as Indie Rock or Techno. This approach would help manage confusions by treating some ambiguities as differences in granularity rather than outright errors. Second, improving segment processing through temporal attention mechanisms or recurrent architectures (LSTMs, Transformers) would make it possible to capture long-term dependencies between segments and significantly increase inter-segment consistency, which is currently limited to 0.1667. Finally, leveraging pretrained architectures such as YAMNet, VGGish, or the Audio Spectrogram Transformer through transfer learning would drastically reduce training time while improving performance, thanks to rich representations learned from millions of audio samples. This would overcome current hardware limitations and enable the exploration of more advanced architectures.

To go further into the technical and coding aspects of the project, it is recommended to consult the project's documentation folder on GitHub. In particular, the documents *strat_project*, *optimisation_documentation*, *project_structure*, as well as the README. You can also look at an execution example in the *execution_example* folder. Examining the performance plots and the automatically generated inference reports can be especially insightful. Please feel free to ask me any question by sending me an email at the following address : colin.manyri@etu.utc.fr

Link to the GitHub Repository:

https://github.com/KolinMTG/FMA_classification